

```

    print(1);                // 调用 AW::print(int)
    print(3.1);              // 调用 Primer::print(double)
    return 0;
}

```

在全局作用域中，函数 `print` 的重载集合包括 `print(int)`、`print(double)` 和 `print(long double)`，尽管它们的声明位于不同作用域中，但它们都属于 `main` 函数中 `print` 调用的候选函数集。

18.2.4 节练习

练习 18.20: 在下面的代码中，确定哪个函数与 `compute` 调用匹配。列出所有候选函数和可行函数，对于每个可行函数的实参与形参的匹配过程来说，发生了哪种类型转换？

```

namespace primerLib {
    void compute();
    void compute(const void *);
}
using primerLib::compute;
void compute(int);
void compute(double, double = 3.4);
void compute(char*, char* = 0);
void f()
{
    . compute(0);
}

```

如果将 `using` 声明置于 `main` 函数中 `compute` 的调用点之前将发生什么情况？重新回答之前的那些问题。

18.3 多重继承与虚继承

多重继承 (multiple inheritance) 是指从多个直接基类 (参见 15.2.2 节, 第 533 页) 中产生派生类的能力。多重继承的派生类继承了所有父类的属性。尽管概念上非常简单, 但是多个基类相互交织产生的细节可能会带来错综复杂的设计问题与实现问题。

为了探讨有关多重继承的问题, 我们将以动物园中动物的层次关系作为教学实例。动物园中的动物存在于不同的抽象级别上。有个体的动物, 如 `Ling-Ling`、`Mowgli` 和 `Balou` 等, 它们以名字进行区分; 每个动物属于一个物种, 例如 `Ling-Ling` 是一只大熊猫; 物种又是科的成员, 大熊猫是熊科的成员; 每个科是动物界的成员, 在这个例子中动物界是指一个动物园中所有动物的总和。

我们将定义一个抽象类 `ZooAnimal`, 用它来保存动物园中动物共有的信息并提供公共接口。类 `Bear` 将存放 `Bear` 科特有的信息, 以此类推。

除了类 `ZooAnimal` 之外, 我们的应用程序还包含其他一些辅助类, 这些类负责封装不同的抽象, 如濒临灭绝的动物。以类 `Panda` 的实现为例, `Panda` 是由 `Bear` 和 `Endangered` 共同派生而来的。

18.3.1 多重继承

在派生类的派生列表中可以包含多个基类：

```
class Bear : public ZooAnimal {
class Panda : public Bear, public Endangered { /* ... */ };
```

每个基类包含一个可选的访问说明符（参见 15.5 节，第 543 页）。一如往常，如果访问说明符被忽略掉了，则关键字 `class` 对应的默认访问说明符是 `private`，关键字 `struct` 对应的是 `public`（参见 15.5 节，第 546 页）。

和只有一个基类的继承一样，多重继承的派生列表也只能包含已经被定义过的类，而且这些类不能是 `final` 的（参见 15.2.2 节，第 533 页）。对于派生类能够继承的基类个数，C++ 没有进行特殊规定；但是在某个给定的派生列表中，同一个基类只能出现一次。

多重继承的派生类从每个基类中继承状态

在多重继承关系中，派生类的对象包含有每个基类的子对象（参见 15.2.2 节，第 530 页）。如图 18.2 所示，在 `Panda` 对象中含有一个 `Bear` 部分（其中又含有一个 `ZooAnimal` 部分）、一个 `Endangered` 部分以及在 `Panda` 中声明的非静态数据成员。

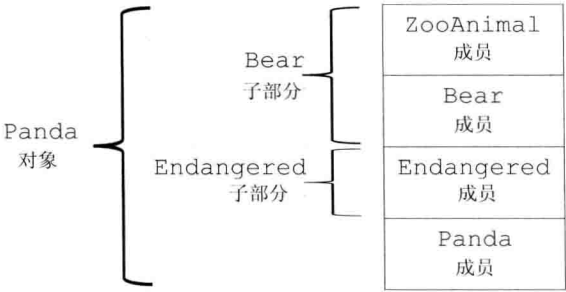


图 18.2: `Panda` 对象的概念结构

派生类构造函数初始化所有基类

构造一个派生类的对象将同时构造并初始化它的所有基类子对象。与从一个基类进行的派生一样（参见 15.2.2 节，第 531 页），多重继承的派生类的构造函数初始值也只能初始化它的直接基类：

```
// 显式地初始化所有基类
Panda::Panda(std::string name, bool onExhibit)
    : Bear(name, onExhibit, "Panda"),
      Endangered(Endangered::critical) { }
// 隐式地使用 Bear 的默认构造函数初始化 Bear 子对象
Panda::Panda()
    : Endangered(Endangered::critical) { }
```

派生类的构造函数初始值列表将实参分别传递给每个直接基类。其中基类的构造顺序与派生列表中基类的出现顺序保持一致，而与派生类构造函数初始值列表中基类的顺序无关。一个 `Panda` 对象按照如下次序进行初始化：

- `ZooAnimal` 是整个继承体系的最终基类，`Bear` 是 `Panda` 的直接基类，`ZooAnimal` 是 `Bear` 的基类，所以首先初始化 `ZooAnimal`。
- 接下来初始化 `Panda` 的第一个直接基类 `Bear`。

- 然后初始化 Panda 的第二个直接基类 Endangered。
- 最后初始化 Panda。

继承的构造函数与多重继承

C++11

在 C++11 新标准中, 允许派生类从它的一个或几个基类中继承构造函数 (参见 15.7.4 节, 第 557 页)。但是如果从多个基类中继承了相同的构造函数 (即形参列表完全相同), 则程序将产生错误:

```
struct Base1 {
    Base1() = default;
    Base1(const std::string&);
    Base1(std::shared_ptr<int>);
};
struct Base2 {
    Base2() = default;
    Base2(const std::string&);
    Base2(int);
};
// 错误: D1 试图从两个基类中都继承 D1::D1(const string&)
struct D1: public Base1, public Base2 {
    using Base1::Base1;           // 从 Base1 继承构造函数
    using Base2::Base2;           // 从 Base2 继承构造函数
};
```

如果一个类从它的多个基类中继承了相同的构造函数, 则这个类必须为该构造函数定义它自己的版本:

805

```
struct D2: public Base1, public Base2 {
    using Base1::Base1;           // 从 Base1 继承构造函数
    using Base2::Base2;           // 从 Base2 继承构造函数
    // D2 必须自定义一个接受 string 的构造函数
    D2(const string &s): Base1(s), Base2(s) { }
    D2() = default;               // 一旦 D2 定义了它自己的构造函数, 则必须出现
};
```

析构函数与多重继承

和往常一样, 派生类的析构函数只负责清除派生类本身分配的资源, 派生类的成员及基类都是自动销毁的。合成的析构函数体为空。

析构函数的调用顺序正好与构造函数相反, 在我们的例子中, 析构函数的调用顺序是 ~Panda、~Endangered、~Bear 和 ~ZooAnimal。

多重继承的派生类的拷贝与移动操作

与只有一个基类的继承一样, 多重继承的派生类如果定义了自己的拷贝/赋值构造函数和赋值运算符, 则必须在完整的对象上执行拷贝、移动或赋值操作 (参见 15.7.2 节, 第 553 页)。只有当派生类使用的是合成版本的拷贝、移动或赋值成员时, 才会自动对其基类部分执行这些操作。在合成的拷贝控制成员中, 每个基类分别使用自己的对应成员隐式地完成构造、赋值或销毁等工作。

例如, 假设 Panda 使用了合成版本的成员 ling_ling 的初始化过程:

```
Panda ying_yang("ying_yang");
```

```
Panda ling_ling = ying_yang;           // 使用拷贝构造函数
```

将调用 Bear 的拷贝构造函数，后者又在执行自己的拷贝任务之前先调用 ZooAnimal 的拷贝构造函数。一旦 ling_ling 的 Bear 部分构造完成，接着就会调用 Endangered 的拷贝构造函数来创建对象相应的部分。最后，执行 Panda 的拷贝构造函数。合成的移动构造函数的工作机理与之类似。

合成的拷贝赋值运算符的行为与拷贝构造函数很相似。它首先赋值 Bear 部分（并且通过 Bear 赋值 ZooAnimal 部分），然后赋值 Endangered 部分，最后是 Panda 部分。移动赋值运算符的工作机理与之类似。

18.3.1 节练习

练习 18.21：解释下列声明的含义，在它们当中存在错误吗？如果有，请指出来并说明错误的原因。

```
(a) class CADVehicle : public CAD, Vehicle { ... };
(b) class Dbllist: public List, public List { ... };
(c) class iostream: public istream, public ostream { ... };
```

练习 18.22：已知存在如下所示的类的继承体系，其中每个类都定义了一个默认构造函数：

```
class A { ... };
class B : public A { ... };
class C : public B { ... };
class X { ... };
class Y { ... };
class Z : public X, public Y { ... };
class MI : public C, public Z { ... };
```

对于下面的定义来说，构造函数的执行顺序是怎样的？

```
MI mi;
```

18.3.2 类型转换与多个基类

在只有一个基类的情况下，派生类的指针或引用能自动转换成一个可访问基类的指针或引用（参见 15.2.2 节，第 530 页；参见 15.5 节，第 544 页）。多个基类的情况与之类似。我们可以令某个可访问基类的指针或引用直接指向一个派生类对象。例如，一个 ZooAnimal、Bear 或 Endangered 类型的指针或引用可以绑定到 Panda 对象上：

```
// 接受 Panda 的基类引用的一系列操作
void print(const Bear&);
void highlight(const Endangered&);
ostream& operator<<(ostream&, const ZooAnimal&);
Panda ying_yang("ying_yang");
print(ying_yang);           // 把一个 Panda 对象传递给一个 Bear 的引用
highlight(ying_yang);       // 把一个 Panda 对象传递给一个 Endangered 的引用
cout << ying_yang << endl; // 把一个 Panda 对象传递给一个 ZooAnimal 的引用
```

编译器不会在派生类向基类的几种转换中进行比较和选择，因为它在看来转换到任何一种基类都一样好。例如，如果存在如下所示的 print 重载形式：

```
void print(const Bear&);  
void print(const Endangered&);
```

则通过 Panda 对象对不带前缀限定符的 print 函数进行调用将产生编译错误:

```
Panda ying_yang("ying_yang");  
print(ying_yang); // 二义性错误
```

基于指针类型或引用类型的查找

807

与只有一个基类的继承一样, 对象、指针和引用的静态类型决定了我们能够使用哪些成员 (参见 15.6 节, 第 547 页)。如果我们使用一个 ZooAnimal 指针, 则只有定义在 ZooAnimal 中的操作是可以使用的, Panda 接口中的 Bear、Panda 和 Endangered 特有的部分都不可见。类似的, 一个 Bear 类型的指针或引用只能访问 Bear 及 ZooAnimal 的成员, 一个 Endangered 的指针或引用只能访问 Endangered 的成员。

举个例子, 已知我们的类已经定义了表 18.1 列出的虚函数, 考虑下面的这些函数调用:

```
Bear *pb = new Panda("ying_yang");  
pb->print(); // 正确: Panda::print()  
pb->cuddle(); // 错误: 不属于 Bear 的接口  
pb->highlight(); // 错误: 不属于 Bear 的接口  
delete pb; // 正确: Panda::~~Panda()
```

当我们通过 Endangered 的指针或引用访问一个 Panda 对象时, Panda 接口中 Panda 特有的部分以及属于 Bear 的部分都是不可见的:

```
Endangered *pe = new Panda("ying_yang");  
pe->print(); // 正确: Panda::print()  
pe->toes(); // 错误: 不属于 Endangered 的接口  
pe->cuddle(); // 错误: 不属于 Endangered 的接口  
pe->highlight(); // 正确: Panda::highlight()  
delete pe; // 正确: Panda::~~Panda()
```

表 18.1: 在 ZooAnimal/Endangered 中定义的虚函数

函数	含有自定义版本的类
print	ZooAnimal::ZooAnimal
	Bear::Bear
	Endangered::Endangered
	Panda::Panda
highlight	Endangered::Endangered
	Panda::Panda
toes	Bear::Bear
	Panda::Panda
cuddle	Panda::Panda
析构函数	ZooAnimal::ZooAnimal
	Endangered::Endangered

18.3.2 节练习

练习 18.23: 使用练习 18.22 的继承体系以及下面定义的类 D, 同时假定每个类都定义了默认构造函数, 请问下面的哪些类型转换是不被允许的?

```
class D : public X, public C { ... };
D *pd = new D;
(a) X *px = pd;      (b) A *pa = pd;
(c) B *pb = pd;      (d) C *pc = pd;
```

练习 18.24: 在第 714 页, 我们使用一个指向 Panda 对象的 Bear 指针进行了一系列调用, 假设我们使用的是一个指向 Panda 对象的 ZooAnimal 指针将发生什么情况, 请对这些调用语句逐一进行说明。

练习 18.25: 假设我们有两个基类 Base1 和 Base2, 它们各自定义了一个名为 print 的虚成员和一个虚析构函数。从这两个基类中我们派生出下面的类, 它们都重新定义了 print 函数:

```
class D1 : public Base1 { /* ... */ };
class D2 : public Base2 { /* ... */ };
class MI : public D1, public D2 { /* ... */ };
```

通过下面的指针, 指出在每个调用中分别使用了哪个函数:

```
Base1 *pb1 = new MI;
Base2 *pb2 = new MI;
D1 *pd1 = new MI;
D2 *pd2 = new MI;
(a) pb1->print();      (b) pd1->print();      (c) pd2->print();
(d) delete pb2;        (e) delete pd1;        (f) delete pd2;
```

18.3.3 多重继承下的类作用域

在只有一个基类的情况下, 派生类的作用域嵌套在直接基类和间接基类的作用域中 (参见 15.6 节, 第 547 页)。查找过程沿着继承体系自底向上进行, 直到找到所需的名字。派生类的名字将隐藏基类的同名成员。

在多重继承的情况下, 相同的查找过程在所有直接基类中同时进行。如果名字在多个基类中都被找到, 则对该名字的使用将具有二义性。

在我们的例子中, 如果我们通过 Panda 的对象、指针或引用使用了某个名字, 则程序会并行地在 Endangered 和 Bear/ZooAnimal 这两棵子树中查找该名字。如果名字在超过一棵子树中被找到, 则该名字的使用具有二义性。对于一个派生类来说, 从它的几个基类中分别继承名字相同的成员是完全合法的, 只不过在使用这个名字时必须明确指出它的版本。



当一个类拥有多个基类时, 有可能出现派生类从两个或更多基类中继承了同名成员的情况。此时, 不加前缀限定符直接使用该名字将引发二义性。

例如, 如果 ZooAnimal 和 Endangered 都定义了名为 max_weight 的成员, 并且 Panda 没有定义该成员, 则下面的调用是错误的:

```
double d = ying_yang.max_weight();
```

Panda 在派生的过程中拥有了两个名为 max_weight 的成员, 这是完全合法的。派生仅仅是产生了潜在的二义性, 只要 Panda 对象不调用 max_weight 函数就能避免二义性错误。另外, 如果每次调用 max_weight 时都指出所调用的版本

◀ 808

◀ 809

(`ZooAnimal::max_weight` 或者 `Endangered::max_weight`), 也不会发生二义性。只有当要调用哪个函数含糊不清时程序才会出错。

在上面的例子中, 派生类继承的两个 `max_weight` 会产生二义性, 这一点显而易见。一种更复杂的情况是, 有时即使派生类继承的两个函数形参列表不同也可能发生错误。此外, 即使 `max_weight` 在一个类中是私有的, 而在另一个类中是公有的或受保护的同样也可能发生错误。最后一种情况, 假如 `max_weight` 定义在 `Bear` 中而非 `ZooAnimal` 中, 上面的程序仍然是错误的。

和往常一样, 先查找名字后进行类型检查 (参见 6.4.1 节, 第 210 页)。当编译器在两个作用域中同时发现了 `max_weight` 时, 将直接报告一个调用二义性的错误。

要想避免潜在的二义性, 最好的办法是在派生类中为该函数定义一个新版本。例如, 我们可以为 `Panda` 定义一个 `max_weight` 函数从而解决二义性问题:

```
double Panda::max_weight() const
{
    return std::max(ZooAnimal::max_weight(),
                    Endangered::max_weight());
}
```

18.3.3 节练习

```
struct Base1 {
    void print(int) const;           // 默认情况下是公有的
protected:
    int ival;
    double dval;
    char cval;
private:
    int *id;
};

struct Base2 {
    void print(double) const;       // 默认情况下是公有的
protected:
    double fval;
private:
    double dval;
};

struct Derived : public Base1 {
    void print(std::string) const;  // 默认情况下是公有的
protected:
    std::string sval;
    double dval;
};

struct MI : public Derived, public Base2 {
    void print(std::vector<double>); // 默认情况下是公有的
protected:
    int *ival;
    std::vector<double> dvec;
};
```

练习 18.26: 已知如上所示的继承体系, 下面对 `print` 的调用为什么是错误的? 适当修改 `MI`, 令其对 `print` 的调用可以编译通过并正确执行。

```
MI mi;
mi.print(42);
```

练习 18.27: 已知如上所示的继承体系, 同时假定为 `MI` 添加了一个名为 `foo` 的函数:

```
int ival;
double dval;
void MI::foo(double cval)
{
    int dval;
    // 练习中的问题发生在此处
}
```

- (a) 列出在 `MI::foo` 中可见的所有名字。
- (b) 是否存在某个可见的名字是继承自多个基类的?
- (c) 将 `Base1` 的 `dval` 成员与 `Derived` 的 `dval` 成员求和后赋给 `dval` 的局部实例。
- (d) 将 `MI::dvec` 的最后一个元素的值赋给 `Base2::fval`。
- (e) 将从 `Base1` 继承的 `cval` 赋给从 `Derived` 继承的 `sval` 的第一个字符。

18.3.4 虚继承

810

尽管在派生列表中同一个基类只能出现一次, 但实际上派生类可以多次继承同一个类。派生类可以通过它的两个直接基类分别继承同一个间接基类, 也可以直接继承某个基类, 然后通过另一个基类再一次间接继承该类。

举个例子, `IO` 标准库的 `istream` 和 `ostream` 分别继承了一个共同的名为 `base_ios` 的抽象基类。该抽象基类负责保存流的缓冲内容并管理流的条件状态。`istream` 是另外一个类, 它从 `istream` 和 `ostream` 直接继承而来, 可以同时读写流的内容。因为 `istream` 和 `ostream` 都继承自 `base_ios`, 所以 `istream` 继承了 `base_ios` 两次, 一次是通过 `istream`, 另一次是通过 `ostream`。

在默认情况下, 派生类中含有继承链上每个类对应的子部分。如果某个类在派生过程中出现了多次, 则派生类中将包含该类的多个子对象。

这种默认的情况对某些形如 `istream` 的类显然是行不通的。一个 `istream` 对象肯定希望在同一个缓冲区中进行读写操作, 也会要求条件状态能同时反映输入和输出操作的情况。假如在 `istream` 对象中真的包含了 `base_ios` 的两份拷贝, 则上述的共享行为就无法实现了。

811

在 C++ 语言中我们通过虚继承 (virtual inheritance) 的机制解决上述问题。虚继承的目的是令某个类做出声明, 承诺愿意共享它的基类。其中, 共享的基类子对象称为虚基类 (virtual base class)。在这种机制下, 不论虚基类在继承体系中出现了多少次, 在派生类中都只包含唯一一个共享的虚基类子对象。

另一个 Panda 类

在过去, 科学界对于大熊猫属于 `Raccoon` 科还是 `Bear` 科争论不休。为了如实地反映这种争论, 我们可以对 `Panda` 类进行修改, 令其同时继承 `Bear` 和 `Raccoon`。此时, 为了避免赋予 `Panda` 两份 `ZooAnimal` 的子对象, 我们将 `Bear` 和 `Raccoon` 继承

ZooAnimal 的方式定义为虚继承。图 18.3 描述了新的继承体系。

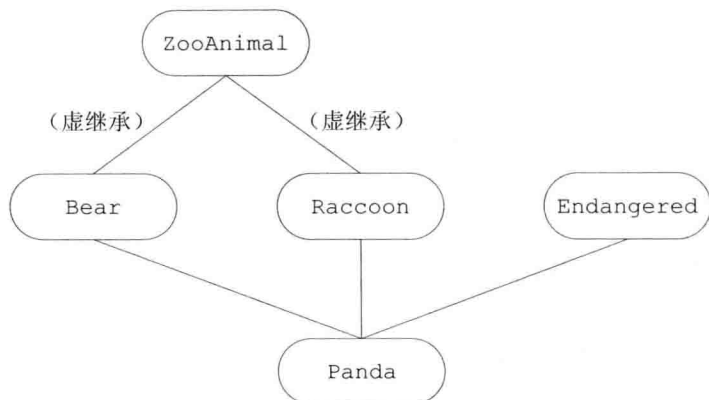


图 18.3: Panda 的虚继承层次

观察这个新的继承体系，我们将发现虚继承的一个不太直观的特征：必须在虚派生的真实需求出现前就已经完成虚派生的操作。例如在我们的类中，当我们定义 Panda 时才出现了对虚派生的需求，但是如果 Bear 和 Raccoon 不是从 ZooAnimal 虚派生得到的，那么 Panda 的设计者就显得不太幸运了。

在实际的编程过程中，位于中间层次的基类将其继承声明为虚继承一般不会带来什么问题。通常情况下，使用虚继承的类层次是由一个人或一个项目组一次性设计完成的。对于一个独立开发的类来说，很少需要基类中的某一个虚基类，况且新基类的开发者也无法改变已存在的类体系。



虚派生只影响从指定了虚基类的派生类中进一步派生出的类，它不会影响派生类本身。

812 使用虚基类

我们指定虚基类的方式是在派生列表中添加关键字 virtual：

```
// 关键字 public 和 virtual 的顺序随意
class Raccoon : public virtual ZooAnimal { /* ... */ };
class Bear : virtual public ZooAnimal { /* ... */ };
```

通过上面的代码我们将 ZooAnimal 定义为 Raccoon 和 Bear 的虚基类。

virtual 说明符表明了一种愿望，即在后续的派生类当中共享虚基类的同一份实例。至于什么样的类能够作为虚基类并没有特殊规定。

如果某个类指定了虚基类，则该类的派生仍按常规方式进行：

```
class Panda : public Bear,
              public Raccoon, public Endangered {
};
```

Panda 通过 Raccoon 和 Bear 继承了 ZooAnimal，因为 Raccoon 和 Bear 继承 ZooAnimal 的方式都是虚继承，所以在 Panda 中只有一个 ZooAnimal 基类部分。

支持向基类的常规类型转换

不论基类是不是虚基类，派生类对象都能被可访问基类的指针或引用操作。例如，下面这些从 Panda 向基类的类型转换都是合法的：

```
void dance(const Bear&);
void rummage(const Raccoon&);
ostream& operator<<(ostream&, const ZooAnimal&);
Panda ying_yang;
dance(ying_yang);           // 正确：把一个 Panda 对象当成 Bear 传递
rummage(ying_yang);        // 正确：把一个 Panda 对象当成 Raccoon 传递
cout << ying_yang;         // 正确：把一个 Panda 对象当成 ZooAnimal 传递
```

虚基类成员的可见性

因为在每个共享的虚基类中只有唯一一个共享的子对象，所以该基类的成员可以被直接访问，并且不会产生二义性。此外，如果虚基类的成员只被一条派生路径覆盖，则我们仍然可以直接访问这个被覆盖的成员。但是如果成员被多余一个基类覆盖，则一般情况下派生类必须为该成员自定义一个新的版本。

例如，假定类 B 定义了一个名为 x 的成员，D1 和 D2 都是从 B 虚继承得到的，D 继承了 D1 和 D2，则在 D 的作用域中，x 通过 D 的两个基类都是可见的。如果我们通过 D 的对象使用 x，有三种可能性：

- 如果在 D1 和 D2 中都没有 x 的定义，则 x 将被解析为 B 的成员，此时不存在二义性，一个 D 的对象只含有 x 的一个实例。
- 如果 x 是 B 的成员，同时是 D1 和 D2 中某一个的成员，则同样没有二义性，派生类的 x 比共享虚基类 B 的 x 优先级更高。
- 如果在 D1 和 D2 中都有 x 的定义，则直接访问 x 将产生二义性问题。

◀ 813

与非虚的多重继承体系一样，解决这种二义性问题最好的方法是在派生类中为成员自定义新的实例。

18.3.4 节练习

练习 18.28: 已知存在如下的继承体系，在 VMI 类的内部哪些继承而来的成员无须前缀限定符就能直接访问？哪些必须有限定符才能访问？说明你的原因。

```
struct Base {
    void bar(int);           // 默认情况下是公有的
protected:
    int ival;
};
struct Derived1 : virtual public Base {
    void bar(char);          // 默认情况下是公有的
    void foo(char);
protected:
    char cval;
};
struct Derived2 : virtual public Base {
    void foo(int);           // 默认情况下是公有的
protected:
    int ival;
```

```

        char cval;
    };
    class VMI : public Derived1, public Derived2 { };

```

18.3.5 构造函数与虚继承

在虚派生中,虚基类是由最低层的派生类初始化的。以我们的程序为例,当创建 Panda 对象时,由 Panda 的构造函数独自控制 ZooAnimal 的初始化过程。

为了理解这一规则,我们不妨假设当以普通规则处理初始化任务时会发生什么情况。在此例中,虚基类将会在多条继承路径上被重复初始化。以 ZooAnimal 为例,如果应用普通规则,则 Raccoon 和 Bear 都会试图初始化 Panda 对象的 ZooAnimal 部分。

当然,继承体系中的每个类都可能在某个时刻成为“最低层的派生类”。只要我们能创建虚基类的派生类对象,该派生类的构造函数就必须初始化它的虚基类。例如在我们的继承体系中,当创建一个 Bear (或 Raccoon) 的对象时,它已经位于派生的最低层,因此 Bear (或 Raccoon) 的构造函数将直接初始化其 ZooAnimal 基类部分:

```

Bear::Bear(std::string name, bool onExhibit):
    ZooAnimal(name, onExhibit, "Bear") { }
Raccoon::Raccoon(std::string name, bool onExhibit)
    : ZooAnimal(name, onExhibit, "Raccoon") { }

```

而当创建一个 Panda 对象时,Panda 位于派生的最低层并由它负责初始化共享的 ZooAnimal 基类部分。即使 ZooAnimal 不是 Panda 的直接基类,Panda 的构造函数也可以初始化 ZooAnimal:

```

Panda::Panda(std::string name, bool onExhibit)
    : ZooAnimal(name, onExhibit, "Panda"),
      Bear(name, onExhibit),
      Raccoon(name, onExhibit),
      Endangered(Endangered::critical),
      sleeping_flag(false) { }

```

虚继承的对象构造方式

含有虚基类的对象的构造顺序与一般的顺序稍有区别:首先使用提供给最低层派生类构造函数的初始值初始化该对象的虚基类子部分,接下来按照直接基类在派生列表中出现的次序依次对其进行初始化。

例如,当我们创建一个 Panda 对象时:

- 首先使用 Panda 的构造函数初始值列表中提供的初始值构造虚基类 ZooAnimal 部分。
- 接下来构造 Bear 部分。
- 然后构造 Raccoon 部分。
- 然后构造第三个直接基类 Endangered。
- 最后构造 Panda 部分。

如果 Panda 没有显式地初始化 ZooAnimal 基类,则 ZooAnimal 的默认构造函数将被调用。如果 ZooAnimal 没有默认构造函数,则代码将发生错误。



虚基类总是先于非虚基类构造，与它们在继承体系中的次序和位置无关。

构造函数与析构函数的次序

一个类可以有多个虚基类。此时，这些虚的子对象按照它们在派生列表中出现的顺序从左向右依次构造。例如，在下面这个稍显杂乱的 TeddyBear 派生关系中有两个虚基类：ToyAnimal 是直接虚基类，ZooAnimal 是 Bear 的虚基类：

815

```
class Character { /* ... */ };
class BookCharacter : public Character { /* ... */ };
class ToyAnimal { /* ... */ };
class TeddyBear : public BookCharacter,
                  public Bear, public virtual ToyAnimal
                  { /* ... */ };
```

编译器按照直接基类的声明顺序对其依次进行检查，以确定其中是否含有虚基类。如果有，则先构造虚基类，然后按照声明的顺序逐一构造其他非虚基类。因此，要想创建一个 TeddyBear 对象，需要按照如下次序调用这些构造函数：

```
ZooAnimal();           // Bear 的虚基类
ToyAnimal();           // 直接虚基类
Character();           // 第一个非虚基类的间接基类
BookCharacter();       // 第一个直接非虚基类
Bear();               // 第二个直接非虚基类
TeddyBear();          // 最低层的派生类
```

合成的拷贝和移动构造函数按照完全相同的顺序执行，合成的赋值运算符中的成员也按照该顺序赋值。和往常一样，对象的销毁顺序与构造顺序正好相反，首先销毁 TeddyBear 部分，最后销毁 ZooAnimal 部分。

18.3.5 节练习

练习 18.29：已知有如下所示的类继承关系：

```
class Class { ... };
class Base : public Class { ... };
class D1 : virtual public Base { ... };
class D2 : virtual public Base { ... };
class MI : public D1, public D2 { ... };
class Final : public MI, public Class { ... };
```

- 当作用于一个 Final 对象时，构造函数和析构函数的执行次序分别是什么？
- 在一个 Final 对象中有几个 Base 部分？几个 Class 部分？
- 下面的哪些赋值运算将造成编译错误？

```
Base *pb;           Class *pc;           MI *pmi;           D2 *pd2;
(a) pb = new Class;           (b) pc = new Final;
(c) pmi = pb;               (d) pd2 = pmi;
```

练习 18.30：在 Base 中定义一个默认构造函数、一个拷贝构造函数和一个接受 int 形参的构造函数。在每个派生类中分别定义这三种构造函数，每个构造函数应该使用它的实参初始化其 Base 部分。

小结

C++语言可以用于解决各种类型的问题，既有几个小时就可以解决的小问题，也有一个大团队工作数年才能解决的超大规模问题。C++的某些特性特别适合于处理超大规模问题，这些特性包括：异常处理、命名空间以及多重继承或虚继承。

异常处理使得我们可以将程序的错误检测部分与错误处理部分分隔开来。当程序抛出一个异常时，当前正在执行的函数暂时中止，开始查找最邻近的与异常匹配的 `catch` 语句。作为异常处理的一部分，如果查找 `catch` 语句的过程中退出了某些函数，则函数中定义的局部变量也随之销毁。

命名空间是一种管理大规模复杂应用程序的机制，这些应用可能是由多个独立的供应商分别编写的代码组合而成的。一个命名空间是一个作用域，我们可以在其中定义对象、类型、函数、模板以及其他命名空间。标准库定义在名为 `std` 的命名空间中。

从概念上来说，多重继承非常简单：一个派生类可以从多个直接基类继承而来。在派生类对象中既包含派生类部分，也包含与每个基类对应的基类部分。虽然看起来很简单，但实际上多重继承的细节非常复杂。特别是对多个基类的继承可能会引入新的名字冲突，并造成来自于基类部分的名字的二义性问题。

如果一个类是从多个基类直接继承而来的，那么有可能这些基类本身又共享了另一个基类。在这种情况下，中间类可以选择使用虚继承，从而声明愿意与层次中虚继承同一基类的其他类共享虚基类。用这种方法，后代派生类中将只有一个共享虚基类的副本。

术语表

捕获所有异常 (catch-all) 异常声明形如 (...) 的 `catch` 子句。一条捕获所有异常的子句可以捕获任意类型的异常。常用于捕获局部检测的异常，该异常将重新抛出到程序的其他部分并最终解决问题。

catch 子句 (catch clause) 程序中负责处理异常的部分。`catch` 子句包含关键字 `catch`，后面是异常声明以及一个语句块。`catch` 子句的代码负责处理异常声明中定义的异常。

构造函数顺序 (constructor order) 在非虚继承中，基类的构造顺序与其在派生列表中出现的顺序一致。在虚继承中，首先构造虚基类。虚基类的构造顺序与其在派生类的派生列表中出现的顺序一致。只有最低层的派生类才能初始化虚基类。虚基类的初始值如果出现在中间基类中，则这些初始值将被忽略。

异常声明 (exception declaration) `catch` 子句中指定其能够处理的异常类型的部分。异常声明的行为与形参列表类似，其中的唯一一个形参通过异常对象进行初始化。如果异常说明符是非引用类型，则异常对象将被拷贝给 `catch`。

异常处理 (exception handling) 管理运行时异常的语言级支持。代码中一个独立开发的部分可以检测并引发异常，由程序的另一个独立开发的部分处理该异常。也就是说，程序的错误检测部分负责抛出异常，而错误处理部分在 `try` 语句块的 `catch` 子句中处理异常。

异常对象 (exception object) 用于在异常的 `throw` 和 `catch` 之间进行通信的对象。在抛出点创建该对象，该对象是被抛出的表达式的副本。在该异常的最后一段处理代码完成之前异常对象都一直存在。异常对象的类型是被抛出的表达式的静态类型。